
UNIT 7 LINKED LISTS

Structure

- 7.0 Introduction
- 7.1 Objectives
- 7.2 Singly Linked Lists
- 7.3 Circularly Linked Lists
- 7.4 Doubly Linked Lists
- 7.5 The Positional List ADT
- 7.6 Sorting a Positional List
- 7.7 Case Study: Maintaining Access Frequencies
- 7.8 Link-Based vs. Array-Based Sequences
- 7.9 Summary
- 7.10 Further Readings

7.0 Introduction

Unlike arrays that store elements in contiguous memory locations, linked lists provide a dynamic way to organize data using nodes. Each node contains both data and a next node reference in the sequence, creating a chain-like structure that can grow or shrink dynamically during program execution.

This unit explores various types of linked lists, including singly linked lists, doubly linked lists, circular linked lists, and the positional list ADT.

7.1 Objectives

After completing this unit, you will be able to:

- Explain the fundamental concepts and components of linked list data structures, including nodes, references, and dynamic memory allocation
- Differentiate between various types of linked lists (singly, doubly, circular) and understand their specific use cases
- Implement list operations including insertion, traversal and deletion.
- Analyze the time and space complexity of various linked list operations
- Compare linked lists with arrays and understand when to choose one over the other
- Design and implement the positional list ADT using a doubly linked list structure
- Debug common linked list problems such as cycle detection and reference management
- Apply linked list concepts to solve real-world programming problems

7.2 Singly Linked List

Singly Linked Lists provides efficient data management and manipulation. Unlike arrays, which have a fixed size and require reshuffling during insertion or deletion, singly linked lists provide dynamic memory allocation, enabling easier modifications. This chapter details the structure, implementation, and applications of singly linked lists in Python.

A Singly Linked List consists of a sequence of nodes, where each node contains two components:

- Data: Node's value
- Next Pointer: Next node's pointer in the sequence.

STRUCTURE OF A NODE

In Python, a node can be represented using a class:

```
class Node:
    def __init__(self, data):
self.data = data # Store the data
self.next = None # Initialize the next pointer to None
'''
```

IMPLEMENTATION OF SINGLY LINKED LISTS

The singly linked list itself can be implemented as follows:

```
class SinglyLinkedList:
    def __init__(self):
self.head = None # Initialize

    def insert_at_end(self, data):
new_node = Node(data)
    if not self.head: # empty list
self.head = new_node
        return
last_node = self.head
        while last_node.next: # last node traversal
last_node = last_node.next
last_node.next = new_node # new node link

    def display(self):
current = self.head
        while current: # Traverse and print each node's data
print(current.data, end=" -> ")
            current = current.next
        print("None") # Indicate the end of the list
'''
```

KEY OPERATIONS ON SINGLY LINKED LISTS

INSERTION

Insertion can occur at various positions within the list:

- At the beginning
- At the end
- At a specific position

Example: The following Python code has three insert operations – insert at the beginning, insert at the end and insert after a node.

```
class SinglyLinkedList:
    def insert_at_beginning(self, data):
new_node = Node(data)
new_node.next = self.head
```

```
self.head = new_node
```

```
def insert_at_end(self, data):
    new_node = Node(data)
    if self.head is None:
        self.head = new_node
    return
```

```
current = self.head
while current.next:
    current = current.next
current.next = new_node
```

```
def insert_after_node(self, prev_node, data):
    if prev_node is None:
        print("Previous node cannot be None")
    return
```

```
new_node = Node(data)
new_node.next = prev_node.next
prev_node.next = new_node
```

DELETION

Deletion operations can also target specific nodes:

- Delete by value
- Delete at a specific position

Example: Deleting by Value

```
def delete_by_value(self, value):
    current = self.head

    if current and current.data == value: # Check if head needs to be deleted
        self.head = current.next # Update head to next node
    return
```

```
prev = None
while current and current.data != value: # Traverse until value is found
    prev = current
    current = current.next
```

```
if not current: # Value not found in list
    return
```

```
prev.next = current.next # Bypass the deleted node
'''
```

Another example of the delete operation is given below. The code supports deletion from the beginning, from the end, and from a specific position.

```
class SinglyLinkedList:
    def delete_from_beginning(self):
```

```

    if self.head is None:
        return None

    temp = self.head
    self.head = self.head.next
    return temp.data

    def delete_from_end(self):
        if self.head is None:
            return None

        if self.head.next is None:
            temp = self.head
            self.head = None
            return temp.data

        current = self.head
        while current.next.next:
            current = current.next

        temp = current.next
        current.next = None
        return temp.data

    def delete_node(self, key):
        if self.head is None:
            return

        if self.head.data == key:
            self.head = self.head.next
            return

        current = self.head
        while current.next and current.next.data != key:
            current = current.next

        if current.next:
            current.next = current.next.next

```

PRACTICAL USE CASES OF SINGLY LINKED LISTS

- Dynamic Memory Allocation: Singly linked lists are ideal for applications where memory utilization needs to be efficient and dynamic.
- Implementing Stacks and Queues: They can serve as foundational structures for more complex data types such as stacks (LIFO) and queues (FIFO).
- Real-time Applications: In scenarios like browser history or undo functionality in applications, singly linked lists allow for quick insertions and deletions.

BEST PRACTICES FOR USING SINGLY LINKED LISTS

- Memory Management: Always ensure that nodes are properly deallocated when they are no longer needed to avoid memory leaks.

- Error Handling: Implement checks for edge cases such as deletion from an empty list or accessing out-of-bounds indices.
- Maintainability: Keep your code modular by separating logic into distinct methods for insertion, deletion, and traversal.
- Handle edge cases: Check for the edge cases such as empty list, single node list, and operations at the beginning or at the end.

Head reference: Always maintain head reference and update the head reference during insert/delete operations.

Time complexity of various operations

The table provides the time complexity of various operations on singly linked list

Operation	Time complexity	Comments
Insert at beginning	$O(1)$	Constant time operation
Insert at end	$O(n)$	Need to traverse to end
Delete from beginning	$O(1)$	Constant time operation
Delete from end	$O(n)$	Need to traverse to end
Search	$O(n)$	May need to traverse entire list
Access by index	$O(n)$	Need to traverse to index

Singly linked lists are versatile data structures that offer significant advantages over traditional arrays in terms of flexibility and efficiency. By understanding their implementation and key operations in Python, you can effectively utilize them in various programming scenarios. As you continue your programming journey, consider how singly linked lists can enhance your applications through dynamic data management capabilities.

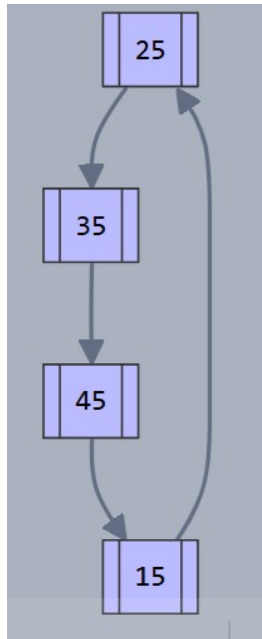
7.3 Circularly Linked List

In the study of data structures, Circularly Linked Lists present a unique variation of linked lists where the last node points back to the first node, forming a circular structure. This configuration allows for more efficient traversal and manipulation of data, particularly in scenarios requiring continuous looping through the list. This chapter will explore the intricacies of circularly linked lists, their implementation in Python, and their practical applications.

A Circularly Linked List is a collection of nodes where each node contains two components:

- Data: The value stored in the node.
- Next Pointer: A reference to the next node in the sequence, with the last node pointing back to the first node.

A visual representation of circularly linked list is given below –



IMPLEMENTATION OF CIRCULARLY LINKED LISTS

The circularly linked list can be implemented as follows:

```

class CircularLinkedList:
    def __init__(self):
        self.head = None # Initialize the head of the list

    def insert_at_end(self, data):
        new_node = Node(data)
        if not self.head: # If the list is empty
            self.head = new_node
            new_node.next = self.head # Point to itself
            return
        last_node = self.head
        while last_node.next != self.head: # Traverse to find the last node
            last_node = last_node.next
        last_node.next = new_node # Link new node at end
        new_node.next = self.head # Point new node back to head

    def display(self):
        if not self.head:
            print("List is empty.")
            return
        current = self.head
        while True: # Traverse until we loop back to head
            print(current.data, end=" -> ")
            current = current.next
            if current == self.head:
                break
        print("(head)")
  
```

KEY OPERATIONS ON CIRCULARLY LINKED LISTS**INSERTION**

Insertion can occur at various positions within a circularly linked list:

- At the beginning
- At the end
- At a specific position

An example code for insert at various positions is given below -

```
class CircularLinkedList:
    def insert_at_beginning(self, data):
        new_node = Node(data)
        if self.is_empty():
            new_node.next = new_node # Points to itself
            self.head = new_node
        else:
            # Check last node
            temp = self.head
            while temp.next != self.head:
                temp = temp.next
            new_node.next = self.head
            temp.next = new_node
            self.head = new_node

    def insert_at_end(self, data):
        new_node = Node(data)
        if self.is_empty():
            new_node.next = new_node
            self.head = new_node
        else:
            temp = self.head
            while temp.next != self.head:
                temp = temp.next
            temp.next = new_node
            new_node.next = self.head

    def insert_after(self, prev_data, data):
        if self.is_empty():
            return

        temp = self.head
        while True:
            if temp.data == prev_data:
                new_node = Node(data)
                new_node.next = temp.next
                temp.next = new_node
                break
            temp = temp.next
            if temp == self.head:
                break
```

ignou
THE PEOPLE'S
UNIVERSITY

DELETION

Deletion operations can target specific nodes by value or position.

Example: Deleting by Value

```
def delete_by_value(self, value):
    if not self.head:
        return "List is empty."

    current = self.head

    if current.data == value: # Check if head needs to be deleted
        if current.next == self.head: # Only one node in list
            self.head = None
            return "Node deleted."

    while current.next != self.head: # Find last node
        current = current.next

    current.next = self.head.next # Bypass head node and update head
    self.head = self.head.next
    return "Node deleted."

prev = None
while current.data != value and current.next != self.head:
    prev = current
    current = current.next

    if current.data == value:
        prev.next = current.next
        return "Node deleted."

    return "Value not found."
'''
```

PRACTICAL USE CASES OF CIRCULARLY LINKED LISTS

- Round Robin Scheduling: In operating systems, circularly linked lists processes scheduling where each process is given an equal share of CPU time in a cyclic manner.
- Multiplayer Games: They can manage players in a round-robin fashion where they can play in turns.
- Music Playlists: Circularly linked lists can create playlists that loop continuously.

BEST PRACTICES FOR USING CIRCULARLY LINKED LISTS

- Memory Management: Deallocate nodes when not required to prevent memory leaks.

- Error Handling: Implement checks for edge cases such as deleting from an empty list or accessing out-of-bounds indices.
- Maintainability: Keep your code modular by separating logic into distinct methods for insertion, deletion, and traversal.

Circularly linked lists provide an efficient way to manage data that requires continuous looping through elements. By understanding their implementation and key operations in Python, you can effectively utilize them in various programming scenarios. As you advance your knowledge of data structures, consider how circularly linked lists can enhance your applications through their dynamic and flexible nature.

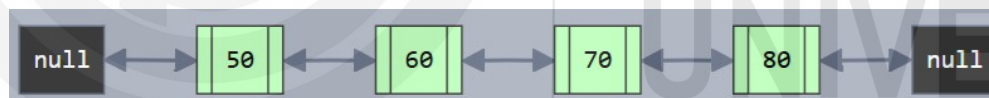
7.4 Doubly linked lists

In the domain of data structures, Doubly Linked Lists offer a more versatile alternative to singly linked lists by allowing traversal in both directions—forward and backward. This capability enhances the efficiency of certain operations, such as deletion and insertion, making doubly linked lists particularly useful in applications requiring bidirectional access to data. This chapter will provide an in-depth exploration of doubly linked lists, their implementation in Python, and their practical applications.

In a Doubly Linked List each node contains three components:

- Data: The value stored in the node.
- Next Pointer: next node's reference in the sequence.
- Previous Pointer: Previous node's reference in the sequence.

A sample doubly linked list is given below -



STRUCTURE OF A NODE

In Python, a node can be represented using a class:

```

class Node:
    def __init__(self, data):
self.data = data # Store the data
self.next = None # Initialize the next pointer to None
self.prev = None # Initialize the previous pointer to None
'''
  
```

IMPLEMENTATION OF DOUBLY LINKED LISTS

The doubly linked list can be implemented as follows:

```

class DoublyLinkedList:
    def __init__(self):
self.head = None # Initialize

    def insert_at_end(self, data):
  
```

Linked List

```
new_node = Node(data)
    if not self.head: # empty list
self.head = new_node
    return
last_node = self.head
    while last_node.next: # Go to last node
last_node = last_node.next
last_node.next = new_node # Link last node to new node
new_node.prev = last_node # Link new node back to last node

def display(self):
    current = self.head
    while current: # Traverse and print each node's data forward
print(current.data, end=" <-> ")
        current = current.next
    print("None") # Indicate the list's end

def display_reverse(self):
    current = self.head
    if not current:
        return "List is empty."

    while current.next: # Traverse to the last node
        current = current.next

    while current: # Traverse and print each node's data backward
print(current.data, end=" <-> ")
        current = current.prev
    print("None") # Indicate the end of the list
...
```

KEY OPERATIONS ON DOUBLY LINKED LISTS

INSERTION

Insertion can occur at various positions within a doubly linked list:

- At the beginning
- At the end
- At a specific position

An example code for insert at various positions is given below -

```
class DoublyLinkedList:
    def insert_at_beginning(self, data):
new_node = Node(data)
        if self.head is None:
self.head = new_node
self.tail = new_node
        else:
new_node.next = self.head
self.head.prev = new_node
self.head = new_node
```

```
def insert_at_end(self, data):
```

```

new_node = Node(data)
    if self.tail is None:
self.head = new_node
self.tail = new_node
    else:
self.tail.next = new_node
new_node.prev = self.tail
self.tail = new_node

def insert_after_node(self, node, data):
    if node is None:
        return

```

```

new_node = Node(data)
new_node.next = node.next
new_node.prev = node

```

```

    if node.next:
node.next.prev = new_node
node.next = new_node

```

```

    if node == self.tail:
self.tail = new_node

```

DELETION

Deletion operations can target specific nodes by value or position.

```

class DoublyLinkedList:
    def delete_from_beginning(self):
        if self.head is None:
            return None

```

```

        temp = self.head
self.head = self.head.next
        if self.head:
self.head.prev = None
        else:
self.tail = None
        return temp.data

```

```

    def delete_from_end(self):
        if self.tail is None:
            return None

```

```

        temp = self.tail
self.tail = self.tail.prev
        if self.tail:
self.tail.next = None
        else:
self.head = None
        return temp.data

```

```

def delete_node(self, node):

```

```

    if node is None:
        return

    if node.prev:
        node.prev.next = node.next
    else:
        self.head = node.next

    if node.next:
        node.next.prev = node.prev
    else:
        self.tail = node.prev

```

PRACTICAL USE CASES OF DOUBLY LINKED LISTS

- Browser History Management: Doubly linked lists are ideal for implementing features like back and forward navigation in web browsers since they allow traversal in both directions.
- Music Playlists: They can manage playlists where users may want to move both forwards and backwards through songs.
- Undo/Redo Functionality: Applications that require undo and redo capabilities can utilize doubly linked lists effectively.

BEST PRACTICES FOR USING DOUBLY LINKED LISTS

- Memory Management: Ensure that nodes are properly deallocated when they are no longer needed to prevent memory leaks.
- Error Handling: Implement checks for edge cases such as deleting from an empty list or accessing out-of-bounds indices.
- Maintainability: Keep your code modular by separating logic into distinct methods for insertion, deletion, and traversal.

Time complexity of various operations

The table provides the time complexity of various operations on doubly linked list

Operation	Time complexity	Comments
Insert at beginning	O(1)	With head pointer
Insert at end	O(1)	With tail pointer
Delete from beginning	O(1)	With head pointer
Delete from end	O(1)	With tail pointer
Search	O(n)	Sequential search
Insert after node	O(1)	With node reference
Delete node	O(1)	With node reference

Doubly linked lists provide enhanced flexibility for managing data that requires bidirectional access. By understanding their implementation and key operations in Python, you can effectively utilize them in various programming scenarios. As

you continue your exploration of data structures, consider how doubly linked lists can improve your applications through their dynamic nature and efficient manipulation capabilities.

7.5 The positional ADT

In the study of data structures, the Positional List Abstract Data Type (ADT) provides a flexible way to manage a sequence of elements where each element can be accessed based on its position. Unlike traditional lists, positional lists allow for efficient insertion and deletion operations at arbitrary positions, making them suitable for applications that require frequent modifications. This chapter will explore the concept of positional lists, their implementation in Python, and practical use cases.

A Positional List is an abstract data type that allows elements to be stored in a sequence while providing direct access to each element by its position. This structure typically supports operations such as insertion, deletion, and retrieval at specified positions.

A positional list is depicted in the below diagram -

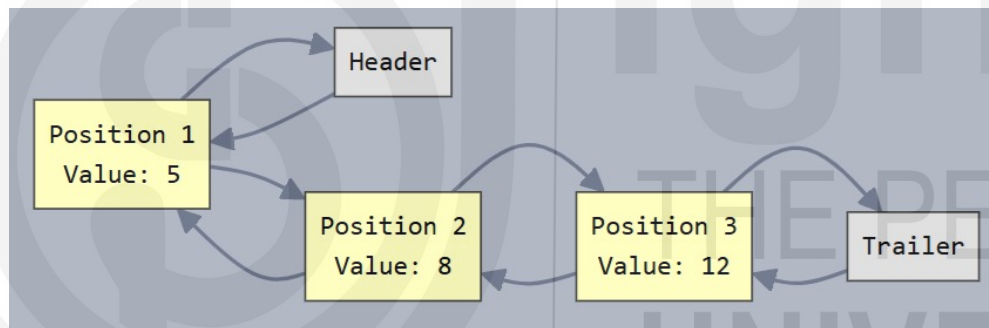


Figure 7.1 : Positional List

STRUCTURE OF A NODE

In Python, a node in a positional list can be represented using a class that contains references to both the next and previous nodes:

```

class Node:
    def __init__(self, data):
self.data = data # Store the data
self.next = None # Pointer to the next node
self.prev = None # Pointer to the previous node
'''
  
```

IMPLEMENTATION OF POSITIONAL LISTS

The positional list can be implemented using a doubly linked list structure where each node maintains pointers to its neighboring nodes:

```

class PositionalList:
    def __init__(self):
  
```

Linked List

```
self.head = None # Initialize the head of the list
self.tail = None # Initialize the tail of the list
self.size = 0    # Track the number of elements

def insert_at_position(self, data, position):
    new_node = Node(data)
    if position < 0 or position > self.size:
        raise IndexError("Position out of bounds.")

    if position == 0: # Insert at the beginning
        if not self.head: # If list is empty
            self.head = new_node
            self.tail = new_node
        else:
            new_node.next = self.head
            self.head.prev = new_node
            self.head = new_node
    elif position == self.size: # Insert at the end
        new_node.prev = self.tail
        if self.tail:
            self.tail.next = new_node
            self.tail = new_node
        else: # Insert in the middle
            current = self.head
            for _ in range(position):
                current = current.next

            new_node.prev = current.prev
            new_node.next = current

            if current.prev:
                current.prev.next = new_node

            current.prev = new_node

    self.size += 1

def delete_at_position(self, position):
    if position < 0 or position >= self.size:
        raise IndexError("Position out of bounds.")

    if position == 0: # Delete from the beginning
        if self.head == self.tail: # Only one element in the list
            self.head = None
            self.tail = None
        else:
            self.head = self.head.next
            if self.head:
                self.head.prev = None

    elif position == self.size - 1: # Delete from the end
        if self.tail:
            self.tail = self.tail.prev
            if self.tail:
```

```

self.tail.next = None

    else: # Delete from the middle
        current = self.head
        for _ in range(position):
            current = current.next

        if current.prev:
            current.prev.next = current.next

        if current.next:
            current.next.prev = current.prev

self.size -= 1

def display(self):
    current = self.head
    while current:
        print(current.data, end=" <-> ")
        current = current.next
    print("None")
...

```

KEY OPERATIONS ON POSITIONAL LISTS

INSERTION AND DELETION

Insertion and deletion operations are required to maintain the integrity of a positional list. The provided methods allow for insertion at any specified position and deletion from any specified position.

Example: Inserting Elements

```

pl = PositionalList()
pl.insert_at_position(10, 0) # Insert at beginning
pl.insert_at_position(20, 1) # Insert at end
pl.insert_at_position(15, 1) # Insert in middle (at index 1)
pl.display()                # Output: 10 <-> 15 <-> 20 <-> None
...

```

Example: Deleting Elements

```

pl.delete_at_position(1)    # Delete element at index 1 (15)
pl.display()                # Output: 10 <-> 20 <-> None
...

```

PRACTICAL USE CASES OF POSITIONAL LISTS

- Text Editor Functionality: Positional lists can be used to implement features such as undo/redo functionality where text can be inserted or deleted at specific positions.
- Dynamic Playlist Management: In music applications, users can add or remove songs from specific positions in their playlists.

- Scheduling Tasks: In task management applications, tasks can be dynamically added or removed based on user input.

BEST PRACTICES FOR USING POSITIONAL LISTS

- Memory Management: Deallocate the nodes when they are no longer required to prevent memory leaks.
- Error Handling: Implement checks for edge cases such as inserting or deleting from an empty list or accessing out-of-bounds indices.
- Maintainability: Keep your code modular by separating logic into distinct methods for insertion, deletion, and traversal.

The Positional List ADT provides a powerful way to manage sequences of elements with efficient insertion and deletion capabilities at arbitrary positions. By understanding its implementation and key operations in Python, you can effectively utilize this data structure in various programming scenarios. As you continue your exploration of data structures, consider how positional lists can enhance your applications through their dynamic nature and flexibility.

7.6 Sorting a positional list

Sorting is a fundamental operation in computer science that arranges the elements of a data structure in a specific order, typically ascending or descending. When dealing with a Positional List, sorting becomes essential for efficiently managing and retrieving data.

SORTING ALGORITHMS OVERVIEW

Several sorting algorithms can be utilized to sort elements within a positional list. The choice of algorithm may depend on factors such as the list size, the nature of the data, and performance requirements. Common sorting algorithms include:

- Bubble Sort
- Insertion Sort
- Selection Sort
- Merge Sort
- Quick Sort

For this chapter, we will focus on implementing Insertion Sort and Merge Sort, as they are particularly well-suited for linked structures like positional lists.

INSERTION SORT FOR POSITIONAL LISTS

Insertion Sort is a good fit for sorting small datasets. Each element is compared one at a time with each new element with those already sorted to build a sorted array.

IMPLEMENTATION OF INSERTION SORT

```

def insertion_sort(positional_list):
    if positional_list.head is None:
        return # List is empty

    sorted_list = PositionalList()
    current = positional_list.head

    while current:
        # Insert current node into sorted_list at the correct position
        insert_sorted(sorted_list, current.data)
        current = current.next

    return sorted_list

def insert_sorted(sorted_list, data):
    new_node = Node(data)
    if not sorted_list.head or sorted_list.head.data >= data: # Insert at head
        new_node.next = sorted_list.head
        if sorted_list.head:
            sorted_list.head.prev = new_node
        sorted_list.head = new_node
    else:
        current = sorted_list.head
        while current.next and current.next.data < data: # Find position to
            insert
            current = current.next
        new_node.next = current.next
        if current.next: # If not inserting at end
            current.next.prev = new_node
        current.next = new_node
        new_node.prev = current
    ``

```

5. MERGE SORT FOR POSITIONAL LISTS

Merge Sort is a divide-and-conquer algorithm that divides the list into halves, sorts each half, and then merges them back together.

IMPLEMENTATION OF MERGE SORT

```

def merge_sort(positional_list):
    if positional_list.size <= 1:
        return positional_list

    mid = positional_list.size // 2
    left_half = PositionalList()
    right_half = PositionalList()

    # Split the original list into two halves
    current = positional_list.head

```

```

        for _ in range(mid):
            left_half.insert_at_end(current.data)
            current = current.next

        while current: # Remaining elements go to right_half
            right_half.insert_at_end(current.data)
            current = current.next

        # Recursively sort both halves and merge them back together
        return merge(merge_sort(left_half), merge_sort(right_half))

def merge(left, right):
    merged = PositionalList()
    left_current = left.head
    right_current = right.head

    while left_current and right_current:
        if left_current.data <= right_current.data:
            merged.insert_at_end(left_current.data)
            left_current = left_current.next
        else:
            merged.insert_at_end(right_current.data)
            right_current = right_current.next

    while left_current: # Append remaining elements from left half
        merged.insert_at_end(left_current.data)
        left_current = left_current.next

    while right_current: # Append remaining elements from right half
        merged.insert_at_end(right_current.data)
        right_current = right_current.next

    return merged
'''

```

PRACTICAL USE CASES FOR SORTING POSITIONAL LISTS

- Data Organization: Sorting is essential in applications where data needs to be presented in an organized manner, such as contact lists or student records.
- Search Optimization: Sorted lists enable faster search operations using binary search techniques.
- Task Management: In task scheduling applications, tasks can be sorted by priority or deadlines to optimize workflow.

BEST PRACTICES FOR SORTING POSITIONAL LISTS

- Choose the Right Algorithm: Depending on the size and nature of your data, select an appropriate sorting algorithm that balances efficiency and simplicity.

- Consider Stability: If maintaining the relative order of equal elements is important, choose a stable sorting algorithm like Merge Sort.
- Optimize Memory Usage: Be mindful of memory consumption when implementing sorting algorithms, especially with large datasets.

Sorting a positional list is required for data retrieval and data management.. By understanding and implementing various sorting algorithms in Python, you can enhance the functionality of your applications significantly. As you continue to explore data structures and algorithms, consider how sorting can improve user experience and application performance through organized data presentation.

7.7 Case study: Maintaining Access Frequencies in positional list

In many applications, it is essential to maintain not only the data itself but also metadata about how frequently each element is accessed. This case study details how to implement access frequency tracking in a Positional List. By doing so, we can optimize operations based on usage patterns, which is particularly useful in scenarios like caching, user activity tracking, and resource management.

BACKGROUND

Access frequency refers to how often an element in a data structure is accessed or modified. We can use the access frequency to improve the performance of the data retrieval by caching the frequently accessed elements.

IMPLEMENTATION OF ACCESS FREQUENCY IN A POSITIONAL LIST

To implement a positional list that maintains access frequencies, we can extend the basic structure of a positional list by adding an additional attribute to each node that tracks its access count.

NODE STRUCTURE WITH ACCESS FREQUENCY

We will modify our existing 'Node' class to include an 'access_count' attribute.

```
class Node:
    def __init__(self, data):
        self.data = data      # Store the data
        self.next = None     # Pointer to the next node
        self.prev = None     # Pointer to the previous node
        self.access_count = 0 # Initialize access count to zero
    ``
```

MODIFIED POSITIONAL LIST CLASS

The 'PositionalList' class will be updated to include methods for accessing elements and incrementing their access counts.

```
class PositionalList:
    def __init__(self):
        self.head = None    # Initialize
        self.tail = None    # Initialize the list's tail
```

```

self.size = 0          # Track elements' number

def insert_at_position(self, data, position):
    new_node = Node(data)
    if position < 0 or position > self.size:
        raise IndexError("Position out of bounds.")

    if position == 0: # Insert at the beginning
        if not self.head: # If list is empty
            self.head = new_node
            self.tail = new_node
        else:
            new_node.next = self.head
            self.head.prev = new_node
            self.head = new_node
    elif position == self.size: # Insert at the end
        new_node.prev = self.tail
        if self.tail:
            self.tail.next = new_node
            self.tail = new_node
        else: # Insert in the middle
            current = self.head
            for _ in range(position):
                current = current.next

        new_node.prev = current.prev
        new_node.next = current

        if current.prev:
            current.prev.next = new_node
        current.prev = new_node

    self.size += 1

def access_element(self, position):
    if position < 0 or position >= self.size:
        raise IndexError("Position out of bounds.")

    current = self.head
    for _ in range(position):
        current = current.next

    current.access_count += 1 # Increment access count on retrieval
    return current.data

def display_access_frequencies(self):
    current = self.head
    while current:
        print(f'Data: {current.data}, Access Count: {current.access_count}')
        current = current.next
    ...

```

PRACTICAL USE CASES FOR MAINTAINING ACCESS FREQUENCIES

- Caching Mechanisms: In systems where data retrieval speed is critical (e.g., web caching), maintaining access frequencies can help to identify the items that should remain in cache and which should be evicted.
- User Behavior Analysis: Applications that track user interactions can be improved by understanding which features or items are accessed most frequently, allowing for better UI/UX design.
- Resource Management: In resource allocation scenarios (e.g., memory management), maintaining access frequencies helps prioritize which resources to keep active based on usage patterns.

BEST PRACTICES FOR ACCESS FREQUENCIES

- Efficient Updates: Ensure that updating access counts does not significantly impact performance, especially in high-frequency access scenarios.
- Memory Management: Regularly clean up nodes that are no longer needed to prevent memory leaks and maintain performance.
- Data Structure Choice: Choose the appropriate underlying data structure based on expected usage patterns and performance requirements.

By implementing access frequency feature in positional lists, applications can become more efficient and responsive to user behavior and system demands. As you explore further into data structures and algorithms, consider how tracking metadata like access frequencies can optimize your applications' performance and user experience.

7.8 Link-Based vs. Array-Based Sequences

Sequences store and manage collections of elements. Two primary implementations of sequences are array-based sequences and link-based sequences (or linked lists). This section will explore these two approaches, examining their strengths and weaknesses in various scenarios.

ARRAY-BASED SEQUENCES

Array-based sequences utilize a contiguous block of memory to store elements. Given below are advantages and disadvantages:

Advantages

- Direct Access: Elements can be accessed in constant time $O(1)$ using their index, making retrieval operations very efficient.
- Cache Performance: Since elements are stored contiguously, array-based sequences benefit from better cache locality, leading to faster access times.

Disadvantages

- Fixed Size: The size of an array must be defined at the time of creation. Resizing an array requires allocating a new larger array and copying elements, which can be time-consuming.
- Inefficient Insertions/Deletions: Inserting or deleting elements (especially in the middle or at the beginning) requires shifting subsequent elements, resulting in $O(n)$ time complexity for these operations.

LINK-BASED SEQUENCES

Link-based sequences, or linked lists, have nodes that point to next and previous nodes. This structure provides a different set of advantages and disadvantages:

Advantages

- Dynamic Size: Linked lists can grow and shrink dynamically as needed without the need for resizing or reallocating memory.
- Efficient Insertions/Deletions: Addition or deletion of nodes can be done in constant time $O(1)$ if the position is known (e.g., at the beginning or end), making these operations much more efficient compared to arrays.

Disadvantages

- No Direct Access: Accessing an element requires traversal from the list's head, leading to $O(n)$ time complexity for retrieval operations.
- Memory Overhead: Each node needs extra memory for storing pointers, which can lead to increased memory usage compared to arrays, especially for small data types.

COMPARATIVE ANALYSIS

The table compares the array-based sequences and link based sequences –

Feature	Array-Based Sequences	Link based sequences
Memory Allocation	Contiguous block	Dynamic allocation (nodes)
Access Time	$O(1)$ (direct access)	$O(n)$ (sequential access)
Insertion/Deletion time	$O(n)$ (requires shifting)	$O(1)$ (if position is known)
Size	Fixed size	Dynamic size
Memory Overhead	Minimal	Higher due to pointers
Cache Performance	Better due to locality	Poorer due to scattered memory

PRACTICAL USE CASES

When to Use Array-Based Sequences

- When the number of elements is known in advance and does not change frequently.
- When fast access times are critical, such as in applications requiring frequent read operations.

When to Use Link-Based Sequences

- When frequent insertions and deletions are expected.
- In applications where the size of the dataset can vary significantly over time.

Both array-based and link-based sequences have their unique advantages and trade-offs. Understanding these differences is crucial for selecting the appropriate data structure based on specific application requirements. Using memory usage, access patterns, we can optimize performance and resource utilization in their applications.

Check Your Progress

- What is the main difference between a singly linked list and a doubly linked list?** a) A singly linked list needs more memory than a doubly linked list b) A doubly linked list can only traverse forward, while a singly linked list traverses both directions c) A singly linked list node contains one reference (next), while a doubly linked list node has two references (next and previous) d) A doubly linked list can only store numeric data, while a singly linked list can store any data type Correct Answer: c)
- In a circular linked list with 5 nodes, what does the 'next' pointer of the last node point to?** a) NULL b) The first node of the list c) The second to last node d) Undefined memory location Correct Answer: b)
- What is the time complexity of inserting a new node at a known position in a doubly linked list?** a) $O(n)$ b) $O(\log n)$ c) $O(1)$ d) $O(n^2)$ Correct Answer: c) $O(1)$
- Which algorithm is commonly used to detect a cycle in a linked list?** a) Binary Search Algorithm b) Floyd's Cycle-Finding Algorithm (Tortoise and Hare) c) Quicksort Algorithm d) Depth-First Search Correct Answer: b) Floyd's Cycle-Finding Algorithm (Tortoise and Hare)
- What is the primary advantage of using a positional list ADT?** a) It consumes less memory than other list implementations b) It provides faster access to elements c) It provides a higher level of abstraction and encapsulation d) It can only store unique elements Correct Answer: c) It provides a higher level of abstraction and encapsulation
- When deleting a node from a doubly linked list in a language without garbage collection, what is essential?** a) Only update the next references b) Only update the previous references c) Update both previous and next references before freeing memory d) Immediately allocate the freed memory to a new node Correct Answer: c) Update both previous and next references before freeing memory
- What is the time complexity of removing the last node in a doubly linked list?** a) $O(1)$ b) $O(n)$ c) $O(\log n)$ d) $O(n^2)$ Correct Answer: a) $O(1)$
- Which scenario would be best suited for a circular linked list?** a) Implementing a stack b) Managing a round-robin scheduling system c) Storing a binary search tree d) Implementing a hash table Correct Answer: b) Managing a round-robin scheduling system

7.9 Summary

Linked lists serve as a cornerstone of data structure implementation, offering flexibility and efficiency for specific operations that arrays cannot match. It needs more memory due to the storage of references and potentially suffer from lack of

random access, their ability to grow dynamically and efficiently handle insertions and deletions at known positions makes them invaluable in many applications.

Through this unit, we've seen how different variations of linked lists can be adapted to solve specific problems. Singly linked lists offer memory efficiency, doubly linked lists provide bidirectional traversal, circular lists enable cyclic operations, and positional lists abstract the implementation details behind a clean interface.

7.10 Further Readings

Books

- "Data Structures and Algorithms in Python" by Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser
- "Python Data Structures and Algorithms" by Benjamin Baka

Online Resources

- [GeeksforGeeks: Linked List Data Structure](#)

